MASTERCLASS SUNUMU

Flutter'da Yerleşim Düzenleri ve Uyarlanabilir Tasarım

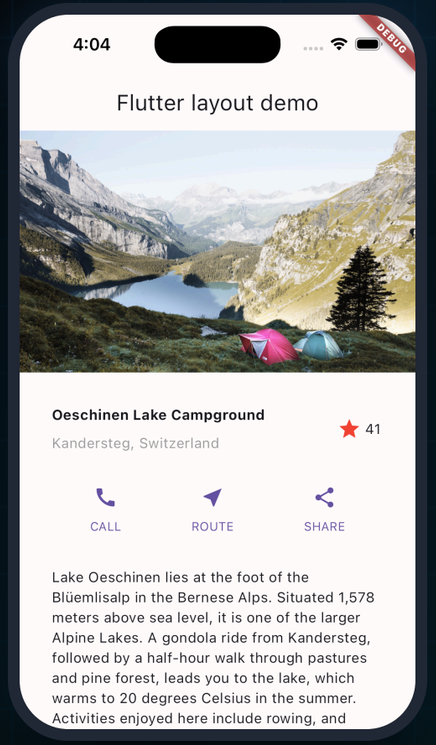
Widget Ağacından Slivers Mimarisine, Çoklu Form Faktörlerinden Platform Adaptasyonuna Kapsamlı Rehber

Modern Layout Mimarisi
Row, Column, Expanded, Stack ve Flex sistemlerinin derinlemesine incelenmesi.

Vaka Analizleri & Uygulama
Strawberry Pavlova kartı ve Lake Oeschinen arayüz inşası metodolojisi.

İleri Kaydırma & Slivers
ListView optimizasyonları, CustomScrollView ve Flow ile 60 FPS Paralaks efekti.

Uyarlanabilir & Büyük Ekranlar
Mobil, katlanabilir cihazlar, tablet ve masaüstü için responsive/adaptive prensipleri.



Flutter ile Üretilen Canlı UI Arayüzü

İçindekiler: Temel Yerleşimler ve Proje İnşası

Bölüm I & Bölüm II Genel Bakış (Sayfa 1 - 40)

01. Flutter'da Yerleşim Felsefesi Slayt 05
'Her şey bir widget'tır' anlayışı, görünür ve görünmeyen düzenleyici öğeler.

02. 3 Altın Kural (Constraints & Sizes) Slayt 06
Constraints go down, sizes go up, parent sets position ilkesi.

03. Eksenler ve Hizalama Slayt 09
MainAxisAlignment ve CrossAxisAlignment ile satır/sütun alan yönetimi.

04. Boyutlandırma & Taşma Çözümleri Slayt 11
Sarı-siyah şeritli overflow hataları, Expanded ve flex çarpanı stratejisi.

05. Vaka Analizi: Strawberry Pavlova Slayt 14
Kompleks iç içe satır/sütun mimarisi ve temiz kod için modüler değişken ayrışımı.

06. Adım Adım Arayüz İnşası (Tutorial) Slayt 17-20
Oeschinen Lake örneği: Title, Button, Text ve ImageSection birleşimi.

İçindekiler: Listeler, Kaydırma ve Slivers

Bölüm III & Bölüm IV Genel Bakış (Sayfa 41 - 80)

07. Liste Türleri ve ListTile Slayt 21-22
Temel ListView, yatay kaydırma ve ScrollConfiguration yapılandırması.

08. Izgara Listeleri: GridView Slayt 23
GridView.count ve GridView.extent ile 2D esnek ızgaralar oluşturma.

09. Heterojen & Aralıklı Listeler Slayt 24-25
Çoklu öğe tipleri ve LayoutBuilder + ConstrainedBox ile eşit aralıklı yapılar.

10. Uzun Listeler & Performans Slayt 26-28
ListView.builder tembel yükleme, prototypeItem ve itemExtent ile layout tasarrufu.

11. Gösterişli Kaydırma & Slivers Slayt 29-31
CustomScrollView, SliverAppBar ve ShrinkWrap tuzağına karşı Slivers üstünlüğü.

12. Flow ile Paralaks Kaydırma Slayt 32-35
Boyama (paint) aşamasına müdahale, ParallaxFlowDelegate ve RenderBox.

SUNUM PLANI // 03

İçindekiler: Uyarlanabilirlik, Girdi ve Platform

Bölüm V & Bölüm VI Genel Bakış (Sayfa 81 - 121)

13. Responsive vs Adaptive Slayt 36
Arayüzü alana sığdırmak (responsive) ile o bağlamda en kullanışlı kılmak (adaptive).

14. Google'ın 3 Adımlı Modeli Slayt 37
Abstract (Soyutla) → Measure (Ölç) → Branch (Dallandır) mimari şablonu.

15. Çoklu Platform Ekosistemi Slayt 38
Android, iOS, Web, Windows, macOS ve Linux üzerinde tek kod tabanı.

16. Büyük Ekranlar & Katlanabilir Cihazlar Slayt 41-43
Letterboxing hatasını önleme, Display API ve NavigationRail adaptasyonu.

17. Çoklu Girdi & Görsel Yoğunluk Slayt 44-46
Fare tekerleği, FocusTraversal, Shortcuts, MouseRegion ve VisualDensity.

18. Platform Kuralları & Best Practices Slayt 47-48
Android vs iOS geçişleri, .adaptive() kurucuları ve altın kurallar.

BÖLÜM 01 // GİRİŞ

01

Flutter Yerleşim Mekanizması & Widget Ağacı

Kullanıcı arayüzünün yapı taşları, hiyerarşik ağaç mimarisi, eksenler ve esnek kutu modelleri

Widget Ağacı Hiyerarşisi

Kökten yaprağa dallanan soyut temsil ve tek sorumluluk ilkesi.

Row ve Column Eksenleri

MainAxisAlignment ve CrossAxisAlignment ile 2D alan dağılımı.

Expanded & Flex Sistemi

Sarı-siyah taşma (overflow) hatalarını önleme stratejileri.

Layout Döngüsü

Constraints down, sizes up, parent sets position mimari kuralı.

Flutter Yerleşim Mekanizmasının 3 Altın Kuralı

'Constraints go down. Sizes go up. Parent sets position.' felsefesi

1. Kısıtlamalar Aşağı İner — Kural 1

Ebeveyn (parent), çocuğuna (child) kullanabileceği min/max genişlik ve yükseklik sınırlarını iletir.

2. Boyutlar Yukarı Çıkar — Kural 2

Çocuk widget, kendine verilen kısıtlamalar çerçevesinde kendi boyutunu (width, height) hesaplayıp yukarı bildirir.

3. Ebeveyn Konumu Belirler — Kural 3

Ebeveyn, çocuğun bildirdiği boyutu alarak onu 2D koordinat düzleminde (dx, dy) konumlandırır.

Tek Geçişli (One-Pass) Render — 120 FPS

Flutter tüm ağacı tek geçişte O(N) karmaşıklıkla hesaplar; bu sayede 120 FPS akıcılık korunur.

Tek Bir Widget'ı Ekrana Yerleştirme

Yerleşim widget'ı seçimi, görünür öğe inşası ve build() metodu

1. Yerleşim Widget'ı Seçin

Öğeyi ortalamak için Center, kenar boşlukları ve arka plan için Container tercih edin.

2. child vs children Parametresi

Tek öğe alan widget'lar child (Center, Container), liste alanlar children alır (Row, Column).

3. Scaffold İskeleti Material

Material uygulamalarında AppBar, arka plan rengi ve Drawer yönetimi için Scaffold kullanılır.

4. Cupertino Desteği Cupertino

iOS tasarım dili için CupertinoApp ve CupertinoPageScaffold kullanılır.

hello_world.dart — DART

```dart
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: const Text('Flutter Layout')),
        body: const Center(
          child: Text(
            'Hello World',
            style: TextStyle(fontSize: 28, color: Colors.white),
          ),
        ),
      ),
    );
  }
}
```

Çoklu Widget Yerleşimi: Row ve Column Temelleri

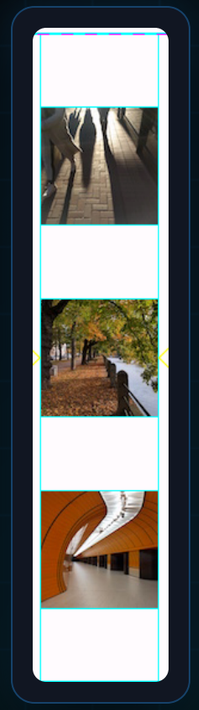
Yatay ve dikey eksenlerde birden fazla elemanı sıralama

Row (Satır)
Altındaki widget'ları soldan sağa yatay eksende yan yana sıralar.

Column (Sütun)
Altındaki widget'ları yukarıdan aşağıya dikey eksende alt alta dizer.

children: <Widget>[] Dizisi
Her iki widget da bir alt öğe dizisi kabul eder; bu öğeler de kendi içlerinde Row veya Column olabilir.

Kullanım Yaygınlığı Temel Taş
Flutter UI dünyasında en sık başvurulan iki temel yerleşim kalıbıdır.



Dikey Eksen (Column) Görsel Öğeleri

Eksenler ve Hizalama: MainAxis & CrossAxis

Satır ve sütunların çalışma yönleri arasındaki kritik geometri

Ana Eksen (Main Axis)
Widget'ın birincil akış yönüdür. Row için yatay, Column için dikeydir.

Çapraz Eksen (Cross Axis)
Ana eksene 90 derece dik olan yöndür. Row için dikey, Column için yataydır.

MainAxisAlignment Enum
start, center, end, spaceBetween, spaceAround, spaceEvenly değerleriyle öğeler dağıtılır.

CrossAxisAlignment Enum
start, center, end, stretch değerleriyle çapraz eksen hizalaması belirlenir.

ÖZELLİK	ROW (SATIR)	COLUMN (SÜTUN)
Ana Akış Yönü	Yatay (Horizontal: X ekseni) Row	Dikey (Vertical: Y ekseni) Column
MainAxisAlignment	Genişlik boyunca öğeleri dağıtır	Yükseklik boyunca öğeleri dağıtır
CrossAxisAlignment	Dikeyde hizalar (üst, orta, alt, stretch)	Yatayda hizalar (sol, orta, sağ, stretch)
Varsayılan Boyut	Genişlikte max, yükseklikte min	Yükseklikte max, genişlikte min

Eşit Aralık ve Boşluk Dağılımı (spaceEvenly)

Boş ekran alanının öğeler arasına ve kenarlara eşit paylaştırılması

spaceEvenly Algoritması

Her elemanın önüne, arasına ve arkasına matematiksel olarak tamamen eşit boşluk koyar.

spaceBetween Alternatifi

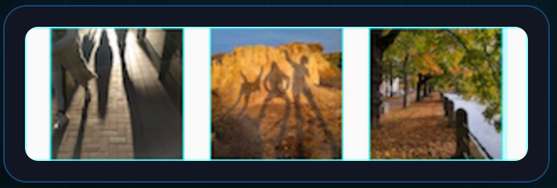
İlk elemanı en başa, son elemanı en sona yaslar; aradaki boşluğu eşit paylaştırır.

spaceAround Alternatifi

Elemanlar arası boşluğun yarısı kadar kenar boşluğu bırakır.

Görsel Varlıklar Asset

Image.asset için pubspec.yaml varlık bildirimleri zorunludur.



Eşit Aralıklarla Hizalanmış 3 Görsel

Boyutlandırma & Taşma Hataları: Expanded & Flex

Sarı-siyah çizgili Overflow hatası ve esnek boyutlandırma çözümü

Taşma (Overflow) Hatası

⚠ A RIGHT OVERFLOWED BY 580 PIXELS

Görsellerin veya metinlerin toplam genişliği ekranı aştığında sarı-siyah şeritli hata belirir ve sağ kenar kesilir.

Expanded ile Esnek Dağılım

✓ ESNEK KOD ÇÖZÜMÜ

flex_solution.dart — DART

```dart
Row(
  crossAxisAlignment: CrossAxisAlignment.center,
  children: [
    Expanded(
      child: Image.asset('images/pic1.jpg'),
    ),
    Expanded(
      flex: 2, // 2 kat daha geniş
      child: Image.asset('images/pic2.jpg'),
    ),
    Expanded(
      child: Image.asset('images/pic3.jpg'),
    ),
  ],
);
```

Elemanları Expanded ile sarmalamak ekran sınırlarına zorlar. flex: 2 katsayısı ile ortadaki görsel iki kat alan kazanır.

Widget'ları Sıkıştırma: MainAxisSize.min

Gereksiz eksen yayılmasını önleyerek bileşenleri bir arada toplama

Varsayılan Davranış: max

Row ve Column varsayılan olarak ana eksende mümkün olan tüm alanı işgal eder.

MainAxisSize.min Kullanımı

Widget'ın yalnızca çocuklarının kapladığı toplam alan kadar yer tutmasını sağlar.

Puanlama & Yıldız İkonları Örnek

Yıldızların birbirinden kopuk dağılması yerine tek bir blok halinde toplanmasını sağlar.

Hiyerarşik Uyum

Diğer üst widget'ların içinde doğal boyutuna kavuşarak esnek kompozisyona izin verir.

packed_row.dart DART

```dart
Row(
  mainAxisSize: MainAxisSize.min,
  children: [
    Icon(Icons.star, color: Colors.green[500]),
    Icon(Icons.star, color: Colors.green[500]),
    Icon(Icons.star, color: Colors.green[500]),
    const Icon(Icons.star, color: Colors.grey),
    const Icon(Icons.star, color: Colors.grey),
  ],
)
```

BÖLÜM 02 // GİRİŞ

02

Vaka Analizleri & Adım Adım Uygulama İnşası

Strawberry Pavlova kartı, modüler mimari, Container kutu modeli ve Lake Oeschinen arayüz inşası

Strawberry Pavlova Kartı

İç içe geçmiş satır ve sütunların gerçek dünya uygulaması.

Modüler Değişken Mimarisi

Büyük widget ağaçlarını bağımsız değişkenlere ve fonksiyonlara ayırma.

Container Kutu Modeli

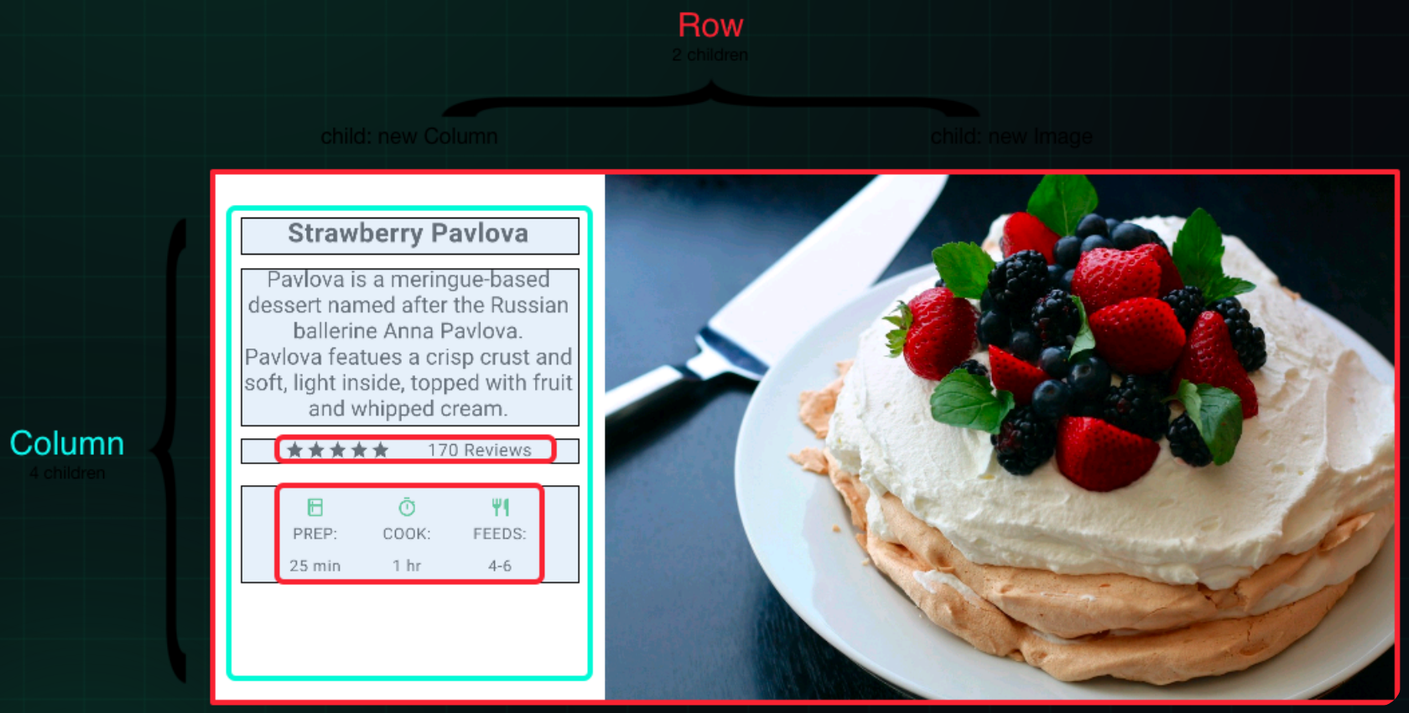
Padding, Margin, Border ve BoxDecoration ile stil katmanı.

Oeschinen Lake Projesi

Başlık, düğmeler, metin ve resim bölümlerinin adım adım inşası.

Strawberry Pavlova: Kompleks Kart Düzeni & Mimari Şema

İç içe geçmiş satır ve sütunların endüstriyel mimari çözümlemesi



Ana Yerleşim (Row) `Row(1:1)`

Solda detayları içeren bir sütun (leftColumn), sağda ise tam boy görsel (mainImage) bulunur.

Puanlama Bölümü (Ratings)

Yıldız ikonlarını barındıran alt Row ve '170 Reviews' metni eşit aralıklarla yerleştirilir.

İkon ve İstatistik Satırı

PREP, COOK ve FEEDS başlıkları ikon + metin sütunları olarak satıra dizilir.

Card & SizedBox Sınırı `Tablet/Landscape`

Arayüz bir Card içine yerleştirilir ve genişliği SizedBox(width: 440) ile sınırlandırılır.

Strawberry Pavlova: İkon ve Bilgi Blokları

PREP, COOK ve FEEDS ikon sütunlarının modüler inşası

İkon Sütun Şablonu

Her bilgi bloğu bir Column içerir: Üstte Icon, ortada başlık metni, altta süre/porsiyon değeri.

spaceEvenly ile Yayılım

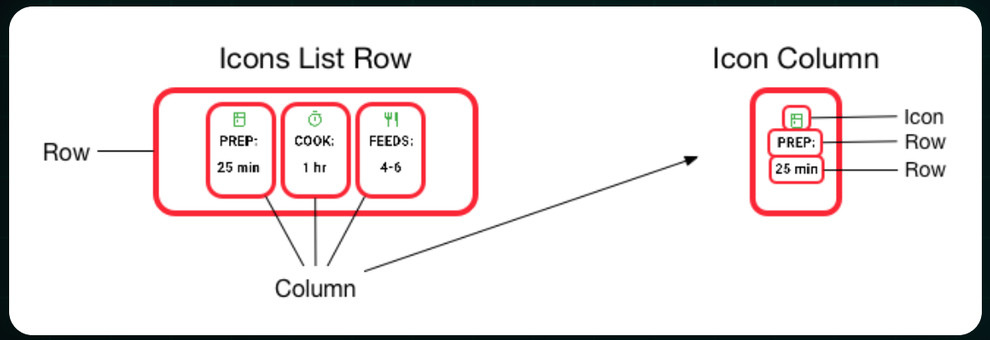
3 ikon sütunu yatay bir Row içine alınarak spaceEvenly ile kutuya dengeli yayılır.

DefaultTextStyle.merge() İpucu

Tüm alt metinlere tek merkezden renk ve punto mirası bırakılarak kod tekrarı engellenir.

Temiz Mimari Ayrımı

Her blok ayrı bir Widget değişkeninde tanımlanarak ana sütunda birleştirilir.



İkon Satırı Detay Şeması

Material Bileşenleri: Card ve ListTile

Hazır semantik yapılar ve derinlik hissi veren kart tasarımları

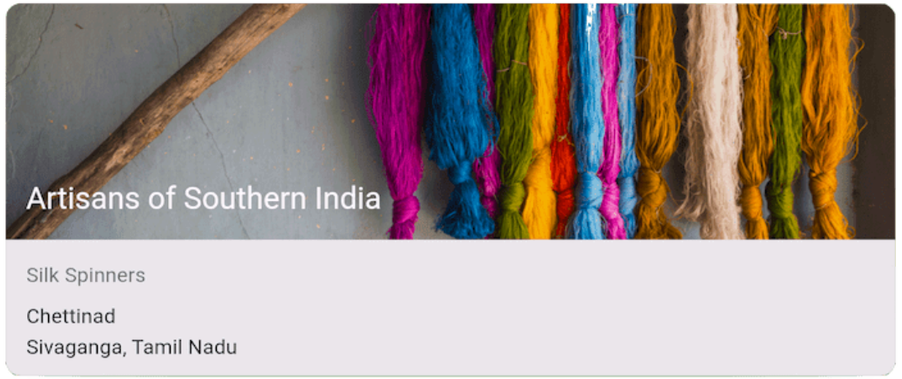
Card Widget'ı
İlgili bilgileri yuvarlatılmış köşeler ve gölgeyle (elevation) sarmalayarak 3B yüzey hissi verir.

elevation Değeri
3D Derinlik

Kartın sanal ışık kaynağından ne kadar yükseleceğini ve gölgenin yayılımını kontrol eder.

ListTile Standardı
leading (baş ikon), title (başlık), subtitle (alt başlık) ve trailing alanlarına sahip hazır satırdır.

Kombinasyon Gücü
Bir Card içinde Column ve Divider ile ayrılmış ListTile'lar kurumsal listeler için idealdir.



Material Card ve ListTile Görünümü

Yerleşim Düzenini Tasarlama: 4 Adımlı İnşa Metodolojisi

'İki kez ölç, bir kez kes' yaklaşımıyla arayüzü analitik bloklara ayrıştırma

1

Temel Eksenleri Çizin

Arayüzün dikey mi yoksa yatay mı aktığını belirleyin. Row ve Column sınırlarını işaretleyin.

Adım 1: Eksenler

→

2

Makro Bloklara Parçalayın

Ekranı bağımsız 4 ana gövdeye bölün: Image, Title, Button, Text bölümleri.

Adım 2: Parçalama

→

3

Modüler Widget'lar Yazın

Her bölümü bağımsız bir StatelessWidget olarak kodlayın. Kendi iç boşluğunu kendi yönetsin.

Adım 3: İzolasyon

→

4

Ana Gövdede Birleştirin

Tüm parçaları tek bir Column altında toplayın ve SingleChildScrollView ile taşmayı önleyin.

Adım 4: Montaj

Başlık ve Düğme Bölümlerinin İnşası (TitleSection & ButtonSection)

Expanded ile başlık alanı esnetme ve ButtonWithText bileşeni

TitleSection Mimarisi

Sol taraftaki metin sütunu Expanded ile sarılarak satırın geri kalanını kaplar; sağda yıldız ve 41 sayısı yer alır.

ButtonWithText Tasarımı

İkon ve altındaki etiketi tek bir Column içinde toplayan tekrar kullanılabilir özel widget.

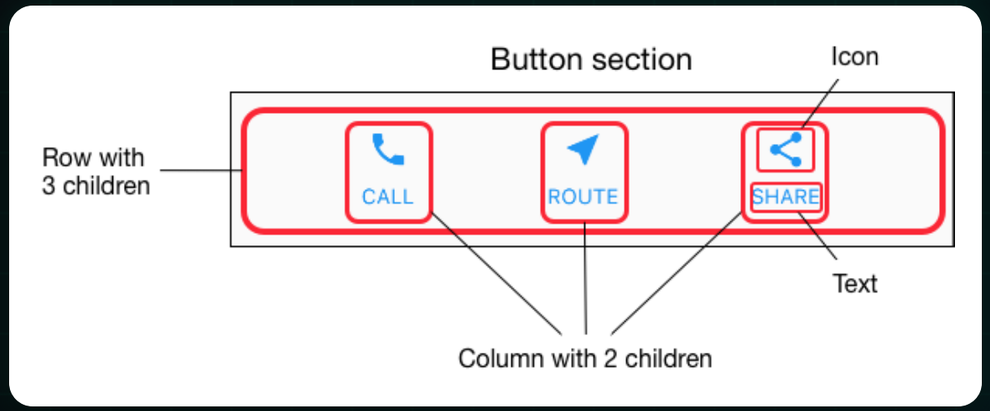
spaceEvenly Düğme Satırı

CALL, ROUTE ve SHARE butonları Row içinde eşit aralıklarla konumlandırılır.

Theme.of(context) Entegrasyonu

Tema

Düğme renkleri uygulamanın birincil renginden (primaryColor) dinamik olarak alınır.



ButtonSection Modüler Şeması

Metin ve Medya: ImageSection, Varlık Yönetimi ve BoxFit

pubspec.yaml yapılandırması, BoxFit.cover ve softWrap kuralı

pubspec.yaml Assets Bildirimi

Görseller proje kökündeki 'images/' dizinine eklenmeli ve pubspec.yaml içinde tanımlanmalıdır.

BoxFit.cover

Görseli orijinal en-boy oranını bozmadan ayırılan render kutusunun tamamını dolduracak şekilde kırpar.

TextSection softWrap

softWrap: true ile metin ekran kenarına ulaştığında kelime sınırlarından düzgünce alt satıra geçer.

SingleChildScrollView Gövdesi

Scroll

Ekran boyutu yetersiz kaldığında taşmayı önlemek için ana sütun kaydırılabilir bir görünüme sarılır.

image_section.dart — DART

```dart
// pubspec.yaml
flutter:
  uses-material-design: true
  assets:
    - images/lake.jpg

// Dart Kodu
class ImageSection extends StatelessWidget {
  const ImageSection({super.key, required this.image});
  final String image;

  @override
  Widget build(BuildContext context) {
    return Image.asset(
      image,
      width: 600,
      height: 240,
      fit: BoxFit.cover,
    );
  }
}
```

Tamamlanan Uygulama: Oeschinen Lake Campground

Modüler bileşenlerin SingleChildScrollView altında kusursuz entegrasyonu

Hiyerarşik Düzen

ImageSection + TitleSection + ButtonSection + TextSection sıralı dikey montajı tamamlandı.

Padding İzolasyonu

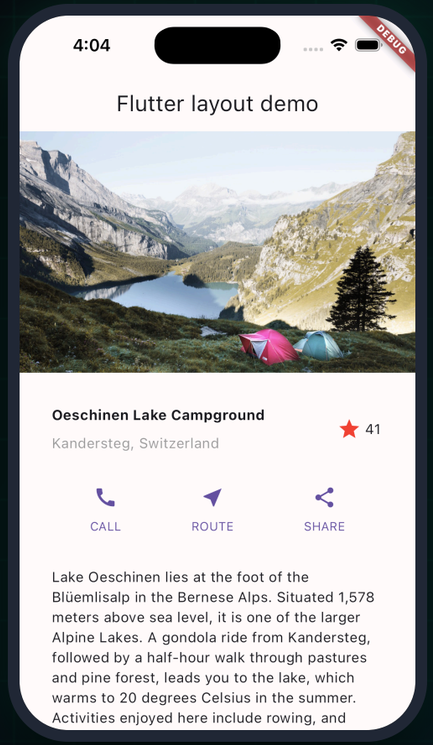
Her bileşen kendi iç boşluğunu yöneterek çevre bileşenlerle çakışmayı engeller.

Hot Reload Avantajı DevEx

Flutter'ın Hot Reload kabiliyeti sayesinde arayüzdeki stil değişiklikleri saniyeler içinde test edilir.

Etkileşim Hazırlığı

Bu statik yerleşim sonraki aşamada StatefulWidget ile beğeni sayacı gibi dinamik özelliklere kavuşabilir.



Tamamlanan Canlı Flutter Uygulaması

BÖLÜM 03 // GİRİŞ

03

İleri Seviye Listeler, Slivers & Paralaks Dinamikleri

ListView tembel yüklemesi, 2D GridView, Slivers kaydırma mimarisi ve Flow widget ile 60 FPS paralaks

ListView & GridView Mimarisi

Temel liste türleri, yatay listeler ve 2D ızgara dağılımları.

Performans & Bellek Yönetimi

ListView.builder tembel yükleme, prototypeItem ve itemExtent tasarrufu.

Slivers Mimarisi (CustomScrollView)

Viewport ve RenderSliver protokolü, SliverAppBar ve elastik kaydırma.

Flow ile Paralaks Efekti

Paint aşamasına müdahale, Matrix4 matris dönüşümleri ve sıfır jank.

Listelerle Çalışma: Temel ListView ve ListTile

Mobil uygulamaların kalbi olan dikey kaydırılabilir veri sunumu

ListView Nedir?

Column benzeri çalışan ancak içeriği ekrana sığmadığında otomatik dikey kaydırma sağlayan widget'tır.

Statik Liste Kullanımı

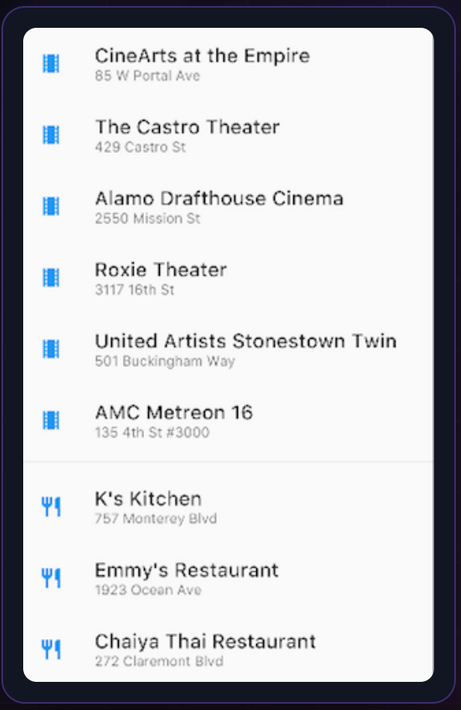
Az sayıda sabit öğe içeren ekranlar için doğrudan ListView(children: []) yapısı idealdir.

ListTile Entegrasyonu

Harita, albüm ve telefon gibi liste girdilerine standart Material görünümü kazandırır.

Divider Bileşeni

Farklı kategorilerdeki liste öğelerini görsel olarak birbirinden ayırmak için kullanılır.



ListTile ve Divider ile Ayrılmış Liste

İki Boyutlu Izgara Listeleri: GridView

GridView.count ve GridView.extent ile 2D esnek veri sunumu

GridView Mimarisi

Verileri iki boyutlu satır ve sütun matriksinde otomatik kaydırılabilir şekilde yerleştirir.

GridView.count

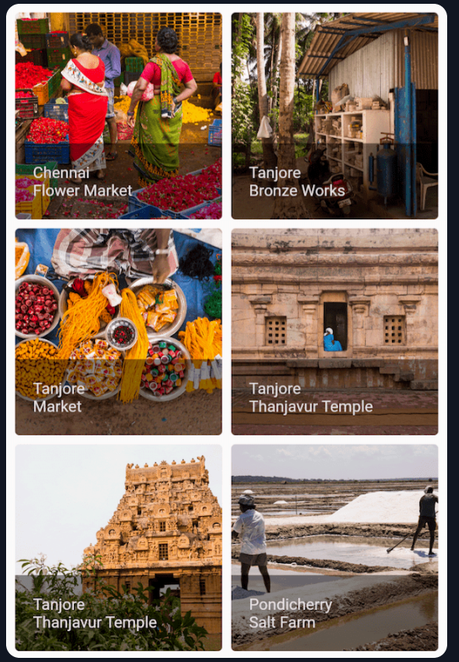
crossAxisCount parametresi ile ekranda kesin olarak kaç sütun bulunacağını sabitler.

GridView.extent Responsive

maxCrossAxisExtent ile her döşemenin azami piksel genişliğini belirler; sütun sayısı ekrana göre dinamik hesaplanır.

Görsel Galerileri

Ürün katalogları ve fotoğraf galerileri için GridView vazgeçilmez bir bileşendir.



GridView.count ile 2D Izgara Görünümü

Heterojen Listeler: Farklı Öğe Türlerini Sıralama

Polimorfizm ve soyut sınıf (abstract class) ile çok tipli listeler

Farklı Veri Tipleri

Aynı liste içinde hem kategori başlığı (HeadingItem) hem mesaj kartı (MessageItem) gösterme.

ListItem Arayüzü

Tüm öğelerin türediği ortak bir ListItem soyut sınıfı ve buildTitle/buildSubtitle metotları.

Dinamik Tip Eşleme (is Kontrolü)

itemBuilder içinde 'item is HeadingItem' kontrolüyle uygun widget'ı render etme.

Temiz Kod Ayrımı

Her öğe tipinin kendi çizim mantığını yönetmesi kod bakımını kolaylaştırır.

mixed_list.dart — DART

```dart
abstract class ListItem {
  Widget build(BuildContext context);
}

class HeadingItem implements ListItem {
  final String heading;
  HeadingItem(this.heading);

  @override
  Widget build(BuildContext context) =>
      Text(heading, style: Theme.of(context).textTheme.headlineSmall);
}

ListView.builder(
  itemCount: items.length,
  itemBuilder: (context, i) => items[i].build(context),
);
```

Aralıklı ve Esnek Liste Öğeleri (Spaced Items)

LayoutBuilder ve ConstrainedBox ile hem eşit boşluk hem kaydırma kabiliyeti

Problem: Spacer vs Scrollable

Kısıtlama

Kaydırılabilir widget'lar içinde Spacer doğrudan çalışmaz çünkü sonlu yükseklik kısıtı yoktur.

1. LayoutBuilder & constraints

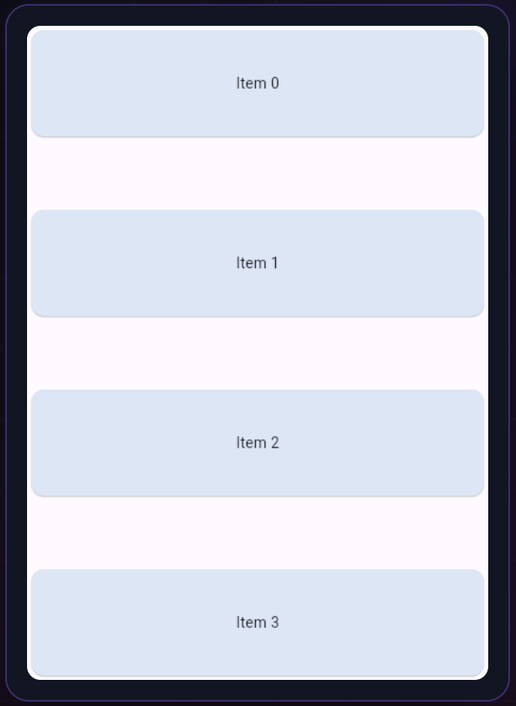
Üst widget'tan gelen BoxConstraints(maxHeight) bilgisini elde etmek için kullanılır.

2. ConstrainedBox(minHeight)

Alt içeriğin en az ekran boyu kadar olmasını garanti eder; böylece spaceBetween düzgün çalışır.

3. IntrinsicHeight

Öğelerin kendi doğal yüksekliklerine göre boyutlanıp kalan boşluğun Spacer ile esnemesini sağlar.



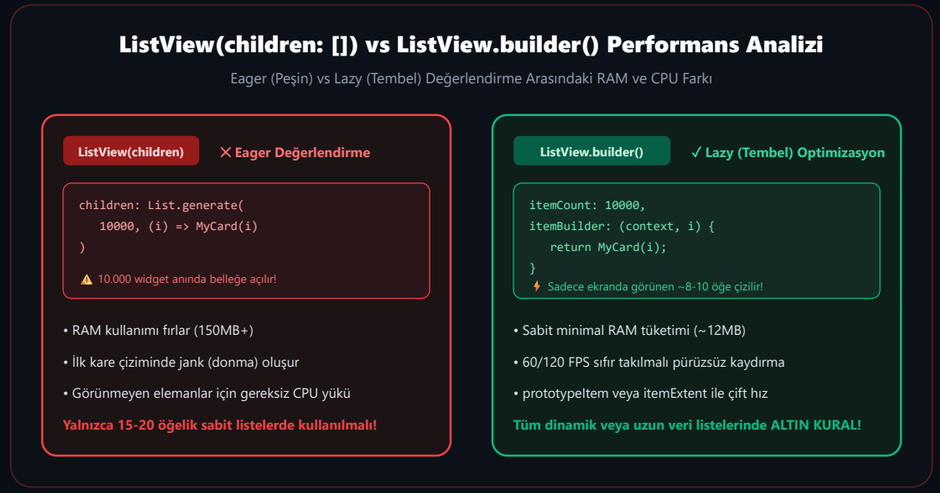
Spaced Items: Küçük Ekranda Kaydırma, Büyükte Eşit Dağılım

Yüksek Performanslı Uzun Listeler: Eager vs Lazy Builder

10.000+ elemanlı listelerde bellek tüketimi ve FPS karşılaştırması

ListView(children: []) Tuzağı

⚠ EAGER / BELLEK PATLAMASI



10.000 elemanın tamamını peşin (eager) belleğe açar. RAM fırlar (150MB+), ilk karede donma (jank) oluşur.

ListView.builder() Çözümü

✓ LAZY / 120 FPS ALTIN STANDART

long_list.dart — DART

```dart
final items = List<String>.generate(
  10000,
  (i) => 'Öğe $i',
);

ListView.builder(
  itemCount: items.length,
  itemBuilder: (context, index) {
    return ListTile(
      title: Text(items[index]),
    );
  },
);
```

Sadece ekranda görünen ~8-10 öğeyi tembel (lazy) render eder. Sabit minimal RAM (~12MB) ve 120 FPS pürüzsüz kaydırma.

Kaydırma Optimizasyonu: Children's Extent

prototypeItem ve itemExtent ile layout hesaplama maliyetini sıfırlama

Layout Hesaplama Yükü

Flutter normalde her liste öğesinin boyutunu tek tek ölçer; ani kaydırmalarda CPU yükü üretir.

Önerilen

itemExtent: Sabit Piksel

Listenizdeki tüm öğeler sabit bir yüksekliğe sahipse (örn. 72px), doğrudan itemExtent: 72.0 verin.

prototypeItem: Şablon Widget

Sabit piksel vermek yerine bir prototip widget vererek boyutun otomatik hesaplanmasını sağlar.

itemExtentBuilder

Her indeksin önceden bilinen değişken bir boyutu varsa hesaplama maliyetini minimize eden geri çağrım.

Özelleşmiş Kaydırma Bileşenleri

Standart listelerin ötesinde zengin etkileşim ve 3D çark görünümleri

Modal

DraggableScrollableSheet

Ekranın altından yukarı sürüklenebilen ve boyutu esneyebilen modal alt sayfalar (bottom sheet).

3D Çark

ListWheelScrollView

Öğeleri 3 boyutlu dönen silindirik bir çark yüzeyine yerleştirir; tarih/saat seçiciler için idealdir.

Scrollbar Widget'ı

Masaüstü ve mobil ekranlarda kaydırma konumunu gösteren ve doğrudan tutulabilen şık çubuk.

ScrollNotification Dinleme

NotificationListener<ScrollNotification> ile kullanıcının kaydırma yönü ve mesafesi milisaniyelik izlenir.

Gösterişli Kaydırma: CustomScrollView & Slivers Protokolü

Viewport ve RenderSliver arasındaki çift yönlü kısıtlama akışı

Sliver Nedir? Protokol

Kaydırılabilir alanın tek bir eksende (scroll axis) dilimlenmiş bağımsız render nesnesidir.

SliverConstraints ↓

Viewport aşağıya scrollOffset, remainingPaintExtent ve crossAxisExtent bilgilerini aktarır.

SliverGeometry ↑

RenderSliver yukarıya paintExtent (görünen alan) ve layoutExtent değerlerini bildirir.

SliverToBoxAdapter Köprü

Klasik kutu modellerini (Container, Row) Slivers dünyasına bağlayan evrensel köprüdür.

Performans Karşılaştırması: ShrinkWrap vs Slivers

İç içe kaydırılabilir yapılarda ölümcül bellek tuzağı ve mimari çözüm

ShrinkWrap Tuzağı Dikkat

shrinkWrap: true, listenin lazy loading kabiliyetini öldürür ve tüm öğeleri tek seferde hesaplatır.

Slivers Mimarisi Doğru Yol

CustomScrollView altındaki tüm sliver'lar tek bir kaydırma kontrolcüsünü paylaşır ve lazy render korunur.

CPU ve Pil Tüketimi

Büyük listelerde shrinkWrap kare atlamalarına (jank) ve cihazın ısınmasına yol açar.

Google Mühendisleri Tavsiyesi

Google ekibi iç içe kaydırmalarda istisnasız Slivers mimarisini tavsiye eder.

ÖZELLİK	SHRİNKWRAP: TRUE TUZAĞI	CUSTOMSCROLLVİEW + SLİVERS
Tembel Yükleme	İptal olur (Tüm elemanlar belleğe alınır) Kötü	Aktif kalır (Yalnızca görünenler çizilir) Mükemmel
Kaydırma Kontrolü	Ayrı ayrı bağımsız controller çakışması	Tek Viewport altında birleşik koordinasyon
Animasyon Desteği	Statik boyutlandırma, efekt desteği yok	Paralaks, esnek başlık, yapışkan öğeler
Büyük Veri Güvenliği	RAM taşması riski (OOM)	Sıfır sızıntı, kesintisiz 120 FPS

Dinamik Başlıklar: SliverAppBar Parametreleri

pinned, floating ve snap kombinasyonlarıyla zengin kullanıcı deneyimi

Sabit

pinned: true

Kullanıcı sayfayı ne kadar aşağı kaydırırsa kaydırsın, başlık çubuğu ekranın en üstünde sabit kalır.

Yüzen

floating: true

Kullanıcı yukarı doğru hafifçe kaydırdığı anda başlık listenin en başına dönmeyi beklemeden anında belirir.

Manyetik

snap: true

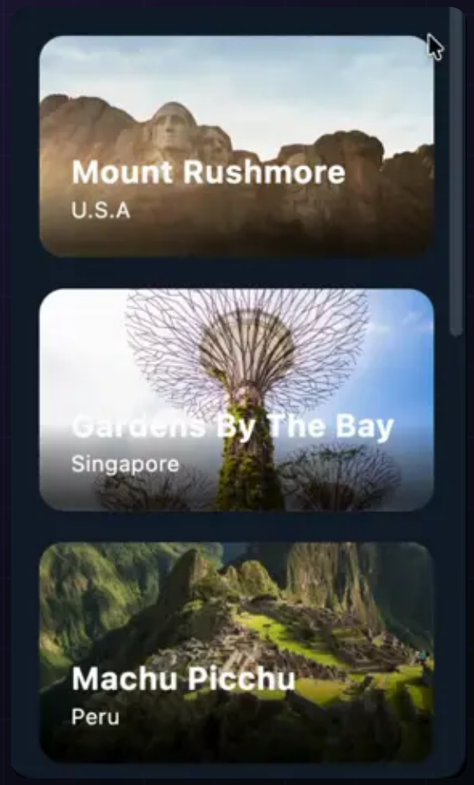
floating ile birlikte kullanılır; kullanıcı parmağını bıraktığı an başlık otomatik olarak ya tamamen açılır ya kapanır.

FlexibleSpaceBar

expandedHeight boyunca arka plan görselini, gradyanı ve başlık metnini pürüzsüzce küçültüp dönüştürür.

Kaydırmalı Paralaks Efekti (Parallax Scrolling)

Arayüz derinliği için katmanların farklı hızlarda kaydırılması



Görsel Derinlik Algısı

Ön plandaki kartlar ile arka plandaki manzara görselleri farklı hızlarda hareket ederek 3B derinlik yaratır.

ListView İçi Kartlar

Her kart kendi içinde bir görsel taşır; liste kaydıkça görsel kartın sınırları içinde ters yönde akar.

Flow Widget ile Sıfır Rebuild

60 FPS

Boyama aşamasında doğrudan GPU matris dönüşümü yapıldığı için widget ağacı yeniden inşa edilmez.

Sayfa 69-80 İncelemesi

Flutter resmi kılavuzundaki 'Scrolling parallax effect' mimarisi adım adım irdelenmektedir.

Flutter Boyama Aşaması (Paint Phase) ve Flow Widget

Layout maliyetini baypas ederek doğrudan RenderObject boyama döngüsüne müdahale

Flutter Render Hattı

Widget → Element → RenderObject (Layout → Paint → Composite).

Flow Widget'ın Farkı

Performans

Çocuklarını boyutlandırırken layout'u bir kez yapar; kaydırma esnasında yalnızca paint() metodunu çağırır.

FlowDelegate Sınıfı

Boyama mantığını üstlenen delegasyon sınıfıdır. paintChildren metodunda her çocuk matrisle çizilir.

Sıfır Jank Garantisi

CPU layout hesaplamasını atladığı için animasyon en yoğun anlarda bile takılmadan akar.

parallax_flow.dart — DART

```dart
Flow(
  delegate: ParallaxFlowDelegate(
    scrollable: Scrollable.of(context),
    listItemContext: context,
    backgroundImageKey: _backgroundImageKey,
  ),
  children: [
    Image.asset(
      'images/lake.jpg',
      key: _backgroundImageKey,
      fit: BoxFit.cover,
    ),
  ],
);
```

ParallaxFlowDelegate: Matematiksel Hesaplama & Matrix4

Global piksel konumundan dikey kaydırma ofsetini türetme

1. Scrollable Boyutunu Alın

ScrollableState.context.findRenderObject() ile kaydırılabilir alanın ekran koordinatlarını yakalayın.

2. Kart Konumunu Ölçün

listItemContext.findRenderObject() üzerinden kartın viewport'taki anlık dikey yüksekliğini bulun.

3. Yüzdelik Oranı Türetin

Formül

scrollPercent = (listItemOffset.dy / (scrollableHeight - listItemHeight)) formülünü işletin.

4. Matrix4.translationValues

Görseli dikey eksende dX = 0, dY = (scrollPercent - 0.5) * maxOffset kadar dönüştürün.

İleri Düzey Paralaks: Özel RenderBox & SingleChildScrollView

Custom scroll context ve RenderProxyBoxMixin ile doğrudan motor entegrasyonu

Doğrudan Render Ağacı

Widget katmanını atlayıp RenderObject seviyesinde layout ve paint döngüsüne müdahale edilir.

Listener Yaşam Döngüsü

attach() ve detach() yaşam döngülerinde scrollable.position dinleyicisi otomatik bağlanıp temizlenir.

performLayout() Kısıtlamaları

Arka plan görseli genişliği doldurmaya zorlanır (BoxConstraints.tightFor).

Maksimum Performans

Ultra Hız

Gereksiz BuildContext yeniden üretimlerini (rebuild) tamamen bertaraf eder.

render_parallax.dart — DART

```dart
class RenderParallax extends RenderBox
    with RenderObjectWithChildMixin<RenderBox>, RenderProxyBoxMixin {
  RenderParallax({required ScrollableState scrollable})
      : _scrollable = scrollable;
  ScrollableState _scrollable;

  @override
  void attach(covariant PipelineOwner owner) {
    super.attach(owner);
    _scrollable.position.addListener(markNeedsLayout);
  }

  @override
  void detach() {
    _scrollable.position.removeListener(markNeedsLayout);
    super.detach();
  }
}
```

BÖLÜM 04 // GİRİŞ

04

Uyarlanabilirlik, Büyük Ekranlar & Platform Adaptasyonu

Çoklu ekran boyut sınıfları, katlanabilir cihazlar, klavye/fare girdileri ve Material 3 vs Cupertino

Responsive vs Adaptive Felsefesi

Arayüzü sığdırmak (responsive) ile o cihaz bağlamında kullanışlı kılmak (adaptive).

Google'ın 3 Adımlı Modeli

Abstract (Soyutla) → Measure (Ölç) → Branch (Dallandır) mimari şablonu.

Büyük Ekran & Katlanabilirler

Window Size Classes, NavigationRail, letterboxing önleme ve Display API.

Çoklu Girdi & Platform Uyumu

Klavye kısayolları, fare imleçleri, VisualDensity ve .adaptive() kurucuları.

Google'ın 3 Adımlı Uyarlama Modeli: Abstract → Measure → Branch

Büyük ölçekli kurumsal uygulamalar için Google mühendislerinin önerdiği tasarım deseni

1

1. Soyutlama (Abstract)

Ekrandaki veri ve navigasyon modellerini görsel widget'lardan soyutlayın. Menü öğelerini tek bir bağımsız listede tanımlayın.

Adım 1: Soyutla

2

2. Ölçme (Measure)

Kullanılabilir ekran alanını MediaQuery.sizeOf(context) veya yerel kısıtlar için LayoutBuilder ile piksel piksel ölçün.

Adım 2: Ölç

3

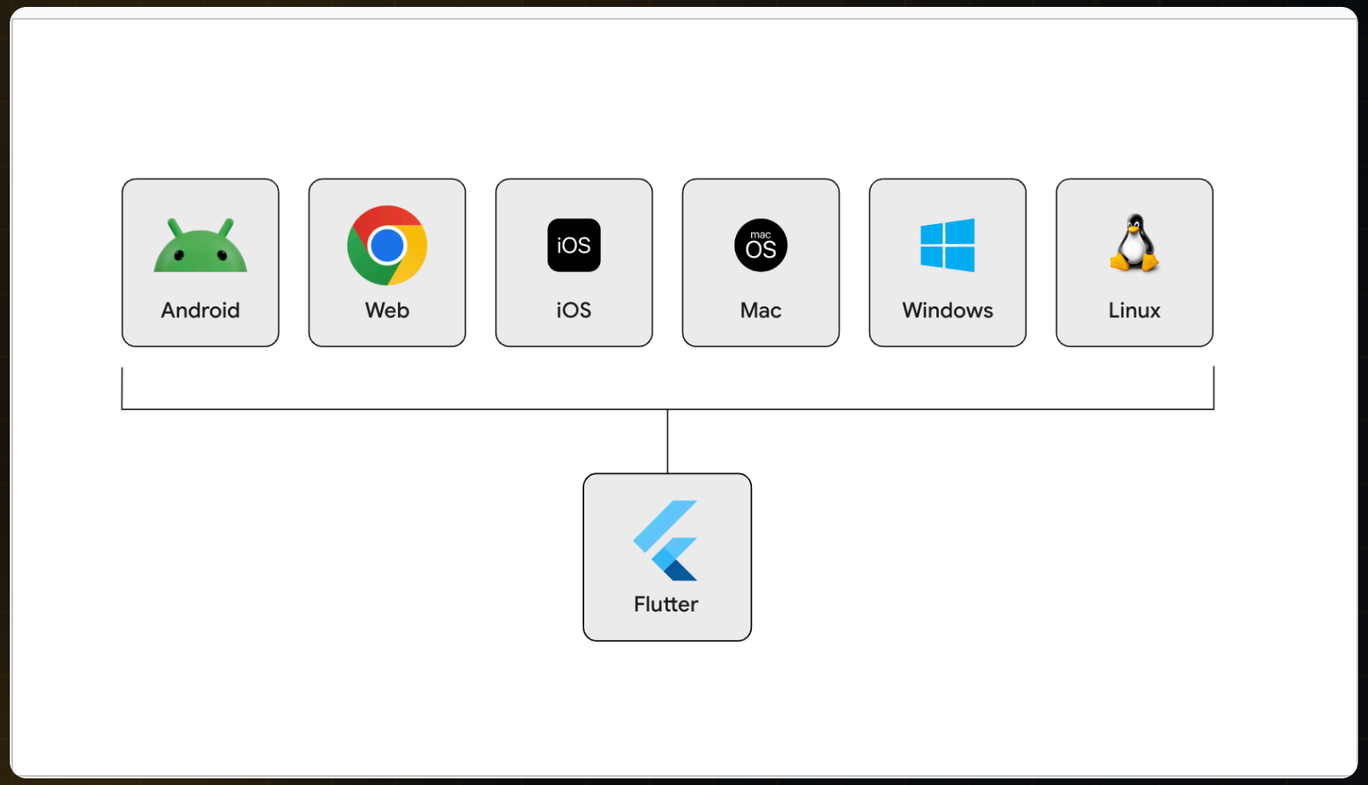
3. Dallandırma (Branch)

600dp kesme noktasına (breakpoint) göre mobilde NavigationBar, tablette NavigationRail şubesine dallandırın.

Adım 3: Dallandır

Çoklu Platform Ekosistemi: Tek Kod Tabanı, 6 Hedef

Android, iOS, Web, Windows, macOS ve Linux üzerinde tek bir arayüz mimarisi



Tek Kod Tabanı Vizyonu — Native

Flutter aynı iş mantığını ve UI kodunu hiçbir emülasyon olmadan yerel makine koduna derler.

Farklı Donanım Beklentileri

Mobil dokunmatik ekran beklerken, masaüstü fare imleci, klavye kısayolları ve çoklu pencere bekler.

Pencere Boyutuna Odaklanın

Cihaz adına değil pencere boyutuna göre karar verin; masaüstü penceresi daraltıldığında mobil düzenine geçebilmelidir.

Google Ekosistem Güvencesi — Resmi

6 platformun tamamı resmi Flutter motoru tarafından birinci sınıf vatandaş olarak desteklenir.

Ekran Ölçümü: MediaQuery.sizeOf vs LayoutBuilder

Neden MediaQuery.of yerine sizeOf kullanılmalı? Rebuild maliyet farkı

MediaQuery.sizeOf(context) Performans

Yalnızca boyut özelliğine abone olur; klavye açılması gibi diğer MediaQuery olaylarında gereksiz rebuild'i önler.

Global Pencere Boyutu

Uygulama penceresinin tamamının mantıksal piksel (logical pixel) cinsinden genişlik ve yüksekliğini verir.

LayoutBuilder: Yerel Kısıtlar

Üst widget'tan o noktaya aktarılan BoxConstraints (min/max aralıkları) bilgisini sağlar.

Doğru Aracı Seçmek

Tüm ekran düzeni için sizeOf; bağımsız bir bileşenin kendi alanı için LayoutBuilder kullanın.

measure_compare.dart — DART

```dart
// Global Pencere Kararı (Sayfa Düzeyi)
final windowSize = MediaQuery.sizeOf(context);
final isLargeScreen = windowSize.width >= 600;

// Yerel Kutu Kararı (Widget Düzeyi)
LayoutBuilder(
  builder: (context, constraints) {
    if (constraints.maxWidth > 400) {
      return TwoColumnLayout();
    }
    return SingleColumnLayout();
  },
);
```

SafeArea ve Donanım Çentikleri (Cutouts)

Kamera delikleri, yuvarlak köşeler ve durum çubuğunun arayüzü kesmesini engelleme

SafeArea Nedir?

Cihazın çentiklerini, kamera deliklerini ve durum çubuklarını otomatik kenar boşluğuna çevirir.

Kenar Bazlı Kontrol

İstenirse top, bottom, left veya right iç boşlukları bağımsız olarak devre dışı bırakılabilir.

Akıllı İç İçe İzolasyon

Zeka

SafeArea altındaki widget'lara yeni bir MediaQuery sunar; böylece iç içe SafeArea'lar çifte boşluk üretmez.

Scaffold.body Sarmalama

Tüm Scaffold'u değil, yalnızca Scaffold'un body alanını sarmalamak AppBar'ın çubuk altına uzanmasını sağlar.

Genişlik Sınıflarına Göre Uyarlanabilir Gezinme (Adaptive Navigation)

Material 3 Window Size Classes: Compact, Medium ve Expanded eşleşmesi

Compact (< 600 dp) Telefon

Akıllı telefonlar için alt NavigationBar; başparmak ergonomisi ön plandadır.

Medium (600 - 840 dp) Tablet

Tablet ve katlanabilir cihazlar için dikey sol NavigationRail; dikey ekran alanını korur.

Expanded (> 840 dp) Masaüstü

Masaüstü ve geniş ekranlar için kalıcı sabit NavigationDrawer ve çok sütunlu Master-Detail düzeni.

Akıcı Geçiş

Pencere yeniden boyutlandırıldığında uygulama bu 3 mod arasında animasyonla pürüzsüzce geçer.

Katlanabilir Cihazlar (Foldables) & Letterboxing Tuzağı

Ekran yönünü kilitlemenin tehlikeleri ve Display API kullanımı

Letterboxing Tuzağı
Kritik Risk

Dikey moda kilitlenen uygulamalar katlanabilir cihaz açıldığında ekran ortasında siyah şeritlerle sıkışır.

Ekran Yönünü Asla Kilitlemeyin!

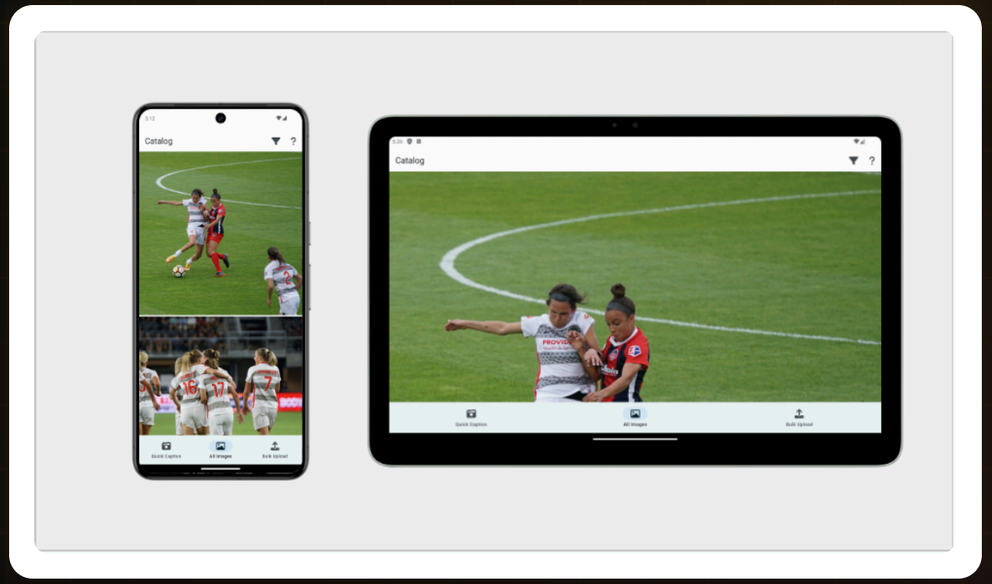
Android ve Flutter yönergeleri dikey kilit (portrait lock) yerine dinamik boyut yönetimini zorunlu kılar.

Display API Entegrasyonu

View.maybeOf(context)?.display üzerinden cihazın piksel oranına ve fiziksel yenileme hızına ulaşılır.

Menteşe (Hinge) Boşlukları

Katlanabilir donanımlarda ekran ortasındaki menteşenin arayüz öğelerini kesmemesi hesaplanmalıdır.



Geniş Ekran ve Katlanabilir Cihaz Düzeni

Uyarlanabilir Gezinme: NavigationBar vs NavigationRail

600dp kesme noktası kuralı: Mobilde alt bar, tablette sol dikey ray

Kullanıcı Ergonomisi

Mobilde başparmak alt kısma erişirken; tablet ve masaüstünde sol yan panel (rail) en doğal alandır.

600dp Standart Breakpoint

600dp Kuralı

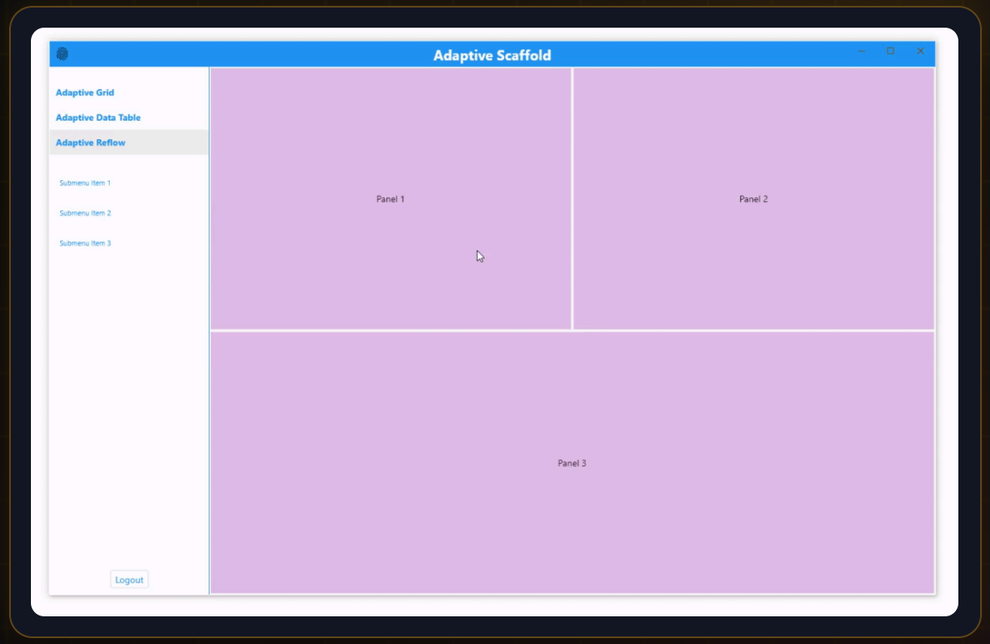
Genişlik < 600dp ise NavigationBar (alt bar); Genişlik >= 600dp ise NavigationRail (sol ray) çizilir.

Ortak Hedef Verisi

Menü öğeleri ve rota indeksleri tek bir liste üzerinden soyutlanarak kod tekrarı önlenir.

Row İçi Entegrasyon

Geniş ekranda NavigationRail solda sabit dururken sağ taraftaki içerik Expanded ile esnetilir.



Adaptive Scaffold: Ekran Boyutuna Göre Otomatik Dönüşüm

Fare ve İşaretçi Etkileşimleri: Listener & MouseRegion

Masaüstü ve web ortamlarında fare tekerleği, hover ve imleç yönetimi

MouseRegion İmleçleri

Tıklanabilir alanlara gelindiğinde cursor: SystemMouseCursors.click ile el simgesi gösterilir.

Hover ve Rollover Efektleri

onEnter ve onExit olayları dinlenerek üzerine gelindiğinde renk veya gölge animasyonları tetiklenir.

Listener ile Tekerlek Yakalama

Tekerlek

onPointerSignal ve PointerScrollEvent ile kaydırma tekerleğinin yönü ve hızı (scrollDelta.dy) kontrol edilir.

Masaüstü Kullanıcı Beklentisi

İmleç geri bildirimi olmayan tıklanabilir elemanlar kullanıcıda arayüzün bozuk olduğu hissini uyandırır.

Klavye ile Gezinme ve Kısayollar (Focus & Shortcuts)

Tab tuşu odak sırası, Shortcuts/Actions yapısı ve donanım klavyesi

FocusTraversalGroup

Kullanıcı Tab tuşuna bastığında odağın hangi sırayla gezeceğini mantıksal gruplara böler.

ReadingOrderTraversalPolicy

Varsayılan okuma sırası politikasıdır; özel politikalarla gezinme sırası tamamen değiştirilebilir.

Kısayol

Shortcuts & Actions

Ctrl+N veya Shift+Enter gibi tuş kombinasyonlarını Intent ve Action yapılarıyla iş mantığına bağlar.

HardwareKeyboard Dinleyicisi

HardwareKeyboard.instance.logicalKeysPressed ile küresel klavye durumlarını anlık kontrol eder.

Görsel Yoğunluk: VisualDensity

Dokunmatik ekranda geniş isabet alanı, masaüstünde kompakt veri sunumu

İsabet Alanı (Hit Area) Farkı

Parmakla dokunmada butonların geniş olması gerekirken, fare imlecinde kompakt öğeler tercih edilir.

VisualDensity Sınıfı

MaterialApp temasındaki görsel yoğunluğu yatay ve dikey eksende negatif veya pozitif değerlerle ayarlar.

Otomatik Pürüzsüz Geçiş

Otomatik

Yoğunluk değiştiğinde destekleyen Material bileşenleri yeni boyutlarına animasyonla pürüzsüzce geçer.

Theme.of(context).visualDensity

Özel widget'larınız da aktif yoğunluk değerini sorgulayarak kendi boyutlarını dinamik uyarlayabilir.

visual_density.dart — DART

```dart
// Dokunmatik modda geniş (0.0), fare modunda kompakt (-1.0)
double densityAmt = touchMode ? 0.0 : -1.0;

VisualDensity density = VisualDensity(
  horizontal: densityAmt,
  vertical: densityAmt,
);

MaterialApp(
  theme: ThemeData(visualDensity: density),
  home: const MainAppScaffold(),
);
```

Platforma Özgü Otomatik Davranışlar: Android vs iOS

Flutter motorunun arka planda işletim sistemine göre devreye aldığı yerel mekanikler

Doğal Platform Hissi

Flutter, geliştirici tek satır yazmasa dahi işletim sisteminin temel beklentilerini otomatik karşılar.

Sayfa Geçişleri (Transitions)

Android'de ZoomPageTransitions, iOS'ta ise yatay kayma ve arka planda paralaks geçişi çalışır.

Kaydırma Fiziği (Physics)

Android'de parıltılı (overscroll glow/stretch), iOS'ta ise yaylanmalı (bouncing) elastik kaydırma.

Tipografi & İkonlar

Android varsayılan olarak Roboto, iOS ise San Francisco yazı tipini ve platforma özel chevron ikonlarını kullanır.

PLATFORM ÖZELLİĞİ	ANDROİD (MATERİAL DESİGN)	İOS (CUPERTİNO HIG)
Sayfa Geçişi	Zoom / Alt-yukarı ölçekleme Android	Yatay Push & Kenardan kaydırıp geri gitme (Edge swipe) iOS
Kaydırma Fiziği	Clamping / Glow parıltısı	Bouncing / Elastik yaylanma ve momentum
En Üste Dönüş	Desteklenmez	Durum çubuğuna dokununca en üste kayma (Return to top)
Metin Seçimi	Tutamaçlı seçim ve Android toolbar	3D Touch kayan imleç ve iOS popover menü

Flutter UI İçin Altın Kurallar & Kapanış Özeti

Yüksek performanslı, ölçeklenebilir ve kusursuz çoklu platform geliştirme ilkeleri

1. Widget'ları Küçük Parçalara Ayırın (Modülerlik) Mimari

Büyük widget ağaçlarını bağımsız fonksiyon/sınıflara bölün ve 'const' kurucular kullanın; rebuild maliyetini sıfırlayın.

2. Doğru Kaydırma Aracını Seçin (Lazy Builder & Slivers) Performans

Uzun listelerde asla ListView(children) veya shrinkWrap kullanmayın; ListView.builder ve Slivers mimarisine sadık kalın.

3. Ekran Yönünü Asla Kilitlemeyin (Dinamik Uyarlanabilirlik) Adaptasyon

Katlanabilir cihazlar, tabletler ve masaüstü için MediaQuery.sizeOf ve Window Size Classes ile serbest kalın.

4. Liste Durumunu Koruyun (PageStorageKey) Kullanıcı Deneyimi

Ekran döndüğünde, pencere yeniden boyutlandığında veya sekmeler değiştiğinde kullanıcının kaydırma pozisyonunu saklayın.